

Movie Recommender Capstone

KJ Monahan

2026-03-07

Introduction

The goal of this project was to create a machine-learning model that can predict how a given user will rate a given movie on a scale of 0.5 stars to 5 stars. These predictions could then be used to recommend new movies to a user.

The dataset contains 10 million ratings of movies by users, as well as the rating timestamp, year released, and genre(s) of the movie. Since users rate varying numbers of movies and movies vary in the number of people who have watched and rated them, the matrix of users and movies is a sparse matrix.

The final model uses a combination of bias effects from known data (user, movie, and genre) and latent factors.

Methods/Analysis

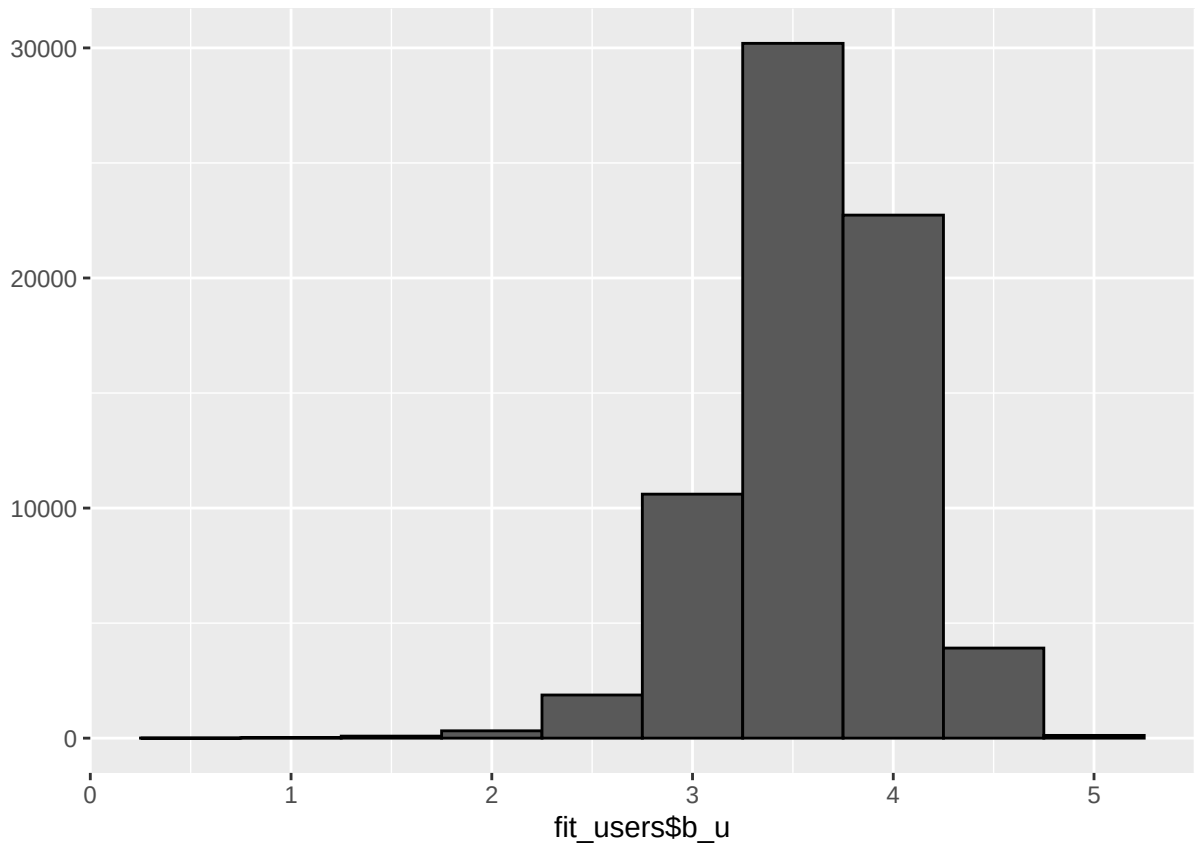
Data Cleaning

Initially, the data contained the movie release year as part of the movie title. After creating my training set and test set (separate from the final holdout test set to use at the end), I cleaned the data by extracting the year into a separate column. My initial hypothesis was that the decade in which the movie was released might have an effect on the outcome, since certain eras of cinema seemed to be more highly regarded than others. So I used the release year to calculate the decade. I also converted the timestamp from a string to a datetime and the genres into factors. I did not end up using all of this data in the model, but I wasn't sure at this point what I would or would not need.

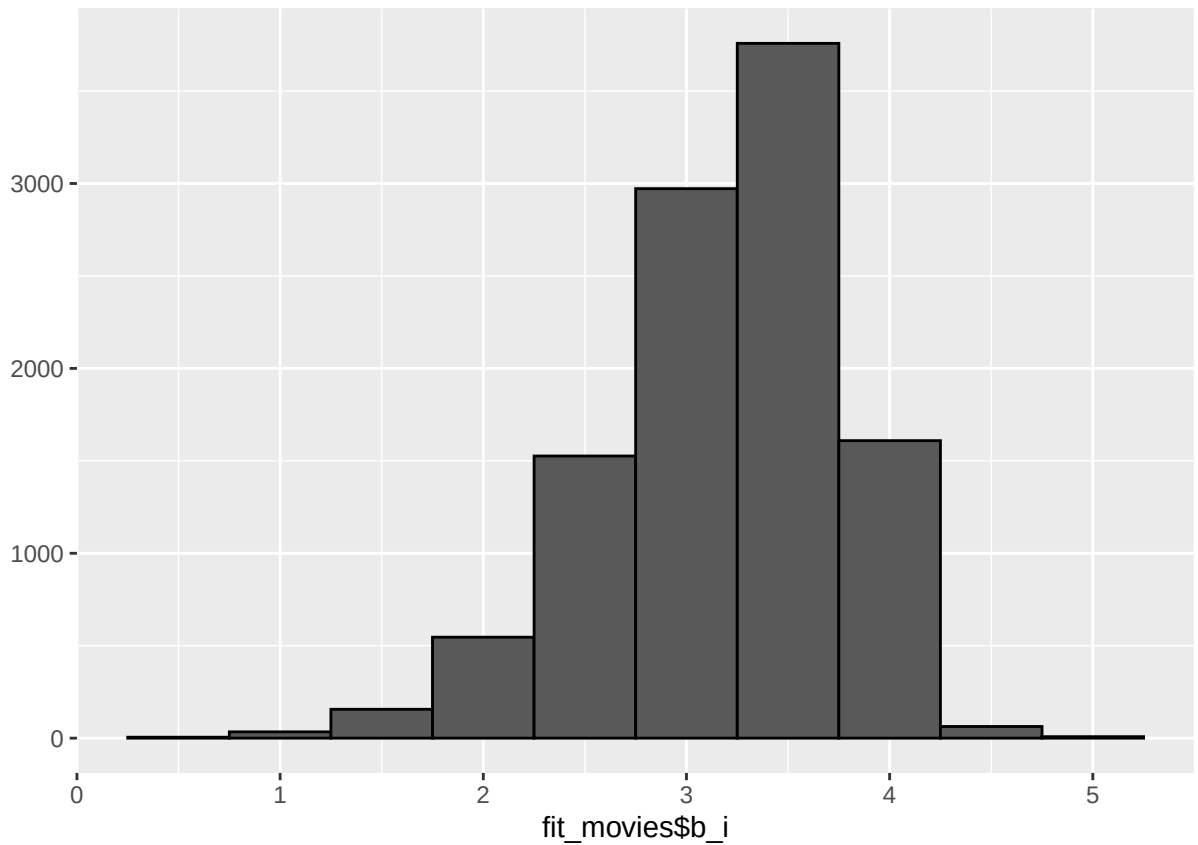
Analysis

I knew from previous work that there would likely be a user effect and a movie effect. To verify this, I grouped ratings by each of those factors and reviewed the results.

```
fit_users <- train_set |>
  group_by(userId) |>
  summarize(b_u = mean(rating))
qplot(fit_users$b_u, bins = 10, color = I("black"))
```



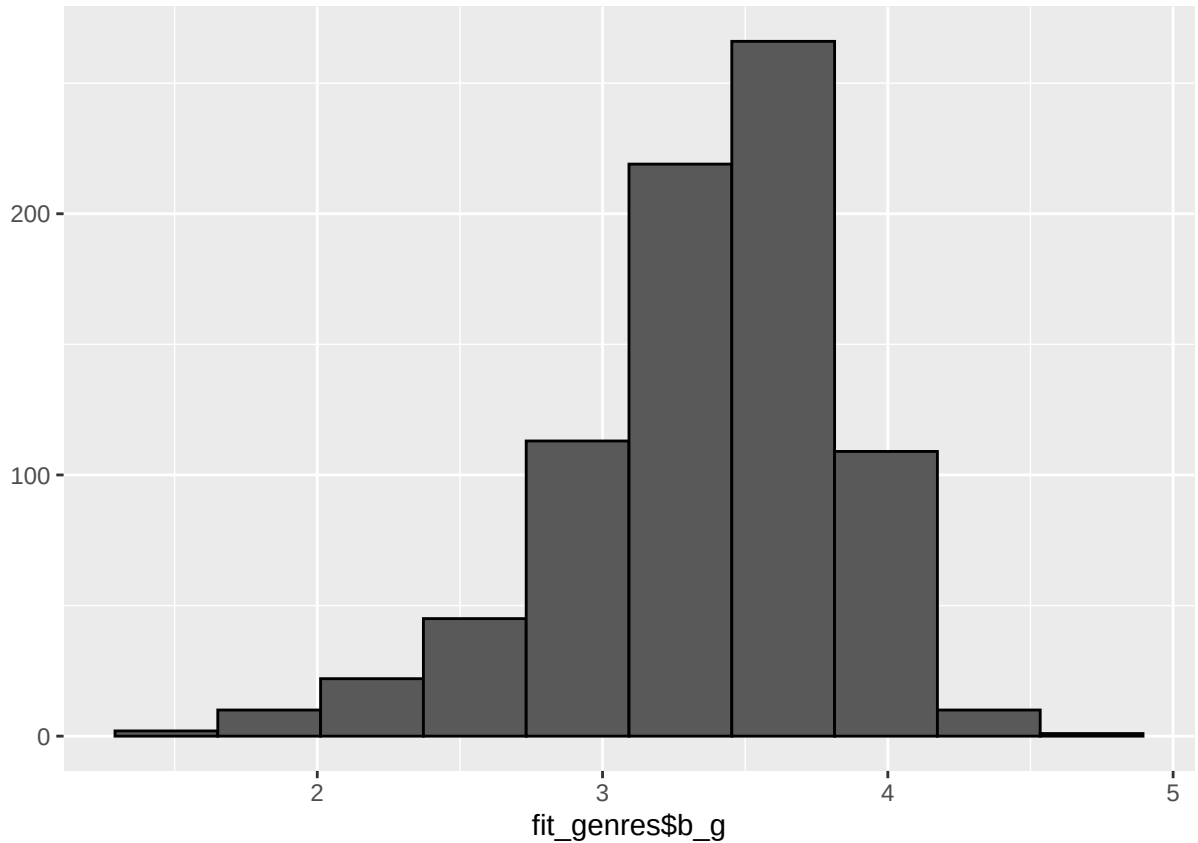
```
fit_movies <- train_set |>
  group_by(movieId) |>
  summarise(b_i = mean(rating))
qplot(fit_movies$b_i, bins = 10, color = I("black"))
```



Ratings did appear to vary by user and by movie.

I expected that ratings would vary by genre, and they did seem to.

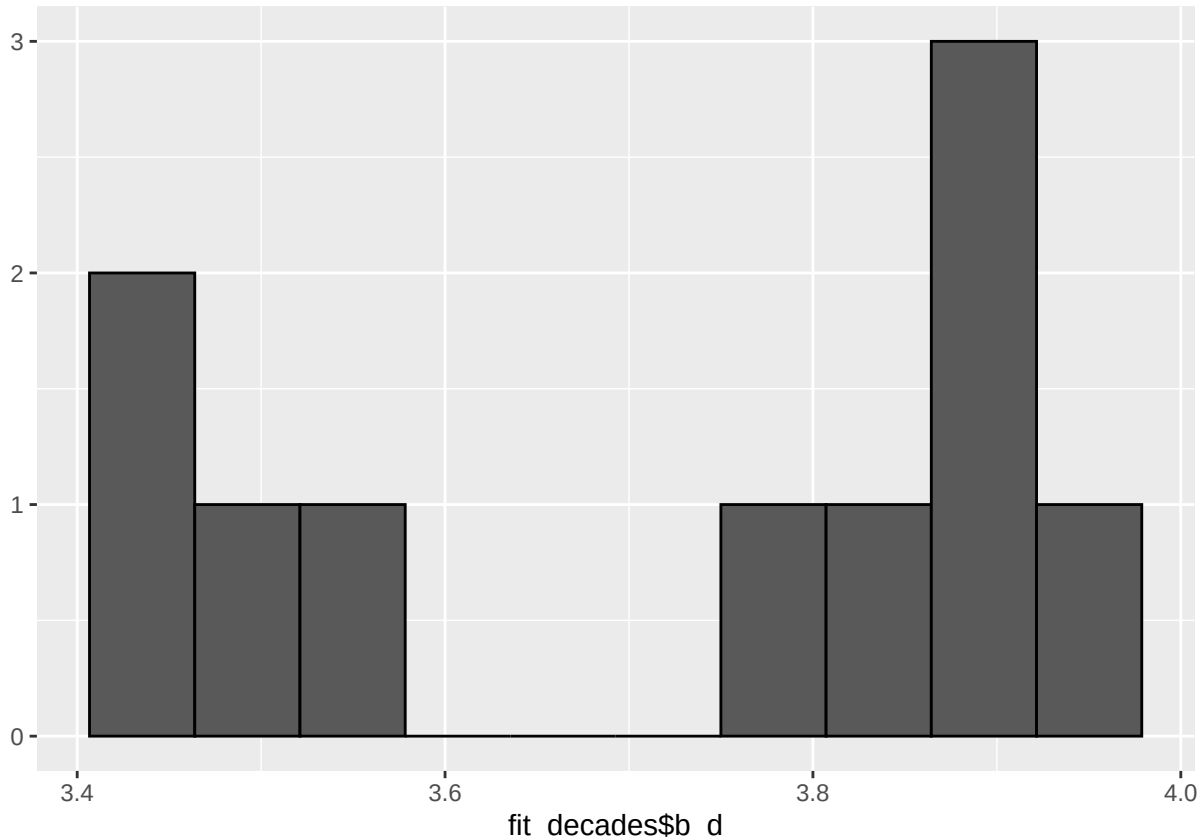
```
fit_genres <- train_set |>
  group_by(genres) |>
  summarise(b_g = mean(rating))
qplot(fit_genres$b_g, bins = 10, color = I("black"))
```



The shape looks a lot like the movie effect, so it is possible that the difference in rating between movies already accounts for genre. However, I thought it still might be useful. I could imagine that knowing that, for instance, “Drama” movies are higher-rated might help give a more accurate prediction for a relatively new drama that didn’t have many ratings for the individual movie yet.

As discussed earlier, I also looked at whether ratings varied by the decade in which the movie was released.

```
fit_decades <- train_set |>
  group_by(decade) |>
  summarise(b_d = mean(rating))
qplot(fit_decades$b_d, bins = 10, color = I("black"))
```



Decades have an interesting shape, clustered between two edges with nothing in the middle. What the graph reveals (although I didn't realize it until later), however, is that the ratings by decade are between 3.4 and a little under 4.0 – a much smaller range than the other factors. The average rating is about 3.5, so while the average rating for a decade can vary from that a little, overall, ratings by decade do not vary enough to make a meaningful impact on the prediction

Baseline: Average Rating

I first calculated the simplest possible model: assuming the average rating for all ratings, so that I would have a starting point to verify that I was improving the accuracy. This proved useful getting any RMSE higher than the baseline was an immediate signal that I had made a mistake in the model and needed to rework it.

```
mu <- mean(train_set$rating, na.rm = TRUE)
rmse(test_set$rating - mu)
```

```
## [1] 1.059728
```

Model 1: Known Effects

I first created a model using the effects of user, movie, and genre. After getting the overall function in place, I created a grid of tuning parameters to find the combination of lambda penalties that gave me the lowest RMSE.

One issue I encountered here was getting the right scale for my lambdas. I initially tried finding a value that was multiplied by the total number of rows. I tested the range 10^{-3} through 10^{-6} . On a training set of roughly 7 million rows, this equated to 7 through 7000. This might have worked, except that I was failing to also account for the different sizes of groups for each factor. As a result, I was either under-regularizing genre effects or so heavily regularizing user effects that their impact was lost. I switched to using constants because

I found that easier to comprehend and then used the mean and median number of ratings per group for each factor to find a more reasonable scale to use for each.

```
fit_als_with_known_effects <- function(data = train_set,
                                       lambda_u = 50,
                                       lambda_m = 100,
                                       lambda_g = 10000,
                                       tol = 1e-6,
                                       max_iter = 500) {

  dt <- as.data.table(copy(data))

  N <- nrow(dt)
  mu <- mean(dt$rating)

  dt[, resid := rating - mu]
  b_i_dt <- b_u_dt <- b_g_dt <- b_d_dt <- NULL
  prev_loss <- rmse(dt$resid)

  for (iter in 1:max_iter) {
    b_i_dt <- dt[, .(b = sum(resid) / (.N + lambda_m)), by = movieId]
    dt[b_i_dt, on = "movieId", bi := i.b]
    dt[, resid := rating - mu - bi]

    b_u_dt <- dt[, .(a = sum(resid) / (.N + lambda_u)), by = userId]
    dt[b_u_dt, on = "userId", bu := i.a]
    dt[, resid := rating - mu - bi - bu]

    b_g_dt <- dt[, .(c = sum(resid) / (.N + lambda_g)), by = genres]
    dt[b_g_dt, on = "genres", bg := i.c]
    dt[, resid := rating - mu - bi - bu - bg]

    loss <- rmse(dt$resid)
    delta <- abs(loss - prev_loss) / prev_loss

    if (delta < tol | delta > 1e07)
      break
    prev_loss <- loss
  }

  list(
    mu = mu,
    b_i = as.data.frame(b_i_dt),
    b_u = as.data.frame(b_u_dt),
    b_g = as.data.frame(b_g_dt)
  )
}

fit <- fit_als_with_known_effects(train_set)
mu <- fit$mu
b_i <- setNames(fit$b_i$b, fit$b_i$movieId)
b_u <- setNames(fit$b_u$a, fit$b_u$userId)
b_g <- setNames(fit$b_g$c, fit$b_g$genres)

resid <- with(test_set, rating - clamp(mu +
```

```

b_i[as.character(movieId)] +
b_u[as.character(userId)] +
b_g[genres]))
rmse(resid)

## [1] 0.9243654

```

This brought my RMSE down quite a bit, but not yet below 0.86.

Latent Factors

I knew not just from the course materials but also from general domain knowledge that there is more that goes into how people rate movies than is captured in the table columns. My two sisters judge movies based on costuming and whether the cinematography is “pretty”. I’ll bail if the world-building doesn’t make sense. I can’t build all of these specific preferences into a model, but it seemed like including latent factors in the model might account for that.

This proved to be the case. While the first part of the final model is identical to the previous version, adding latent factors improved the accuracy of the predictions enough that I could move on to testing it with the final holdout test set.

```

fit_als_with_latent <- function(data = train_set,
                                K = 24,
                                lambda_u = 50,
                                lambda_m = 100,
                                lambda_g = 10000,
                                lambda_pq = 5,
                                tol = 1e-6,
                                max_iter = 500) {
  dt <- as.data.table(copy(data))

  N <- nrow(dt)
  mu <- mean(dt$rating)

  dt[, resid := rating - mu]
  b_i_dt <- b_u_dt <- b_g_dt <- b_d_dt <- NULL
  prev_loss <- rmse(dt$resid)

  for (iter in 1:max_iter) {
    b_i_dt <- dt[, .(b = sum(resid) / (.N + lambda_m)), by = movieId]
    dt[b_i_dt, on = "movieId", bi := i.b]
    dt[, resid := rating - mu - bi]

    b_u_dt <- dt[, .(a = sum(resid) / (.N + lambda_u)), by = userId]
    dt[b_u_dt, on = "userId", bu := i.a]
    dt[, resid := rating - mu - bi - bu]

    b_g_dt <- dt[, .(c = sum(resid) / (.N + lambda_g)), by = genres]
    dt[b_g_dt, on = "genres", bg := i.c]
    dt[, resid := rating - mu - bi - bu - bg]

    loss <- rmse(dt$resid)
    delta <- abs(loss - prev_loss) / prev_loss

    if (delta < tol | delta > 1e07)

```

```

    break
    prev_loss <- loss
  }

prev_loss <- rmse(dt$resid)

unique_users <- unique(dt$userId)
unique_movies <- unique(dt$movieId)
n_users <- length(unique_users)
n_movies <- length(unique_movies)

user_int <- match(dt$userId, unique_users)
movie_int <- match(dt$movieId, unique_movies)
resids <- dt$resid

order_by_user <- order(user_int)
users_by_userId <- user_int[order_by_user]
movies_by_userId <- movie_int[order_by_user]
resids_by_userId <- resids[order_by_user]

order_by_movie <- order(movie_int)
users_by_movieId <- user_int[order_by_movie]
movies_by_movieId <- movie_int[order_by_movie]
resids_by_movieId <- resids[order_by_movie]

u_rle <- rle(users_by_userId)
user_endpoints <- cumsum(u_rle$lengths)
user_starting_points <- c(1L, user_endpoints[-length(user_endpoints)] + 1L)
m_rle <- rle(movies_by_movieId)
movie_endpoints <- cumsum(m_rle$lengths)
movie_starting_points <- c(1L, movie_endpoints[-length(movie_endpoints)] + 1L)

p <- matrix(rnorm(n_users * K, mean = 0, sd = 0.01),
            nrow = n_users,
            ncol = K)
q <- matrix(rnorm(n_movies * K, mean = 0, sd = 0.01),
            nrow = n_movies,
            ncol = K)

lambda_pq_diag <- lambda_pq * diag(K)

for (iter in 1:max_iter) {
  for (i in 1:n_users) {
    current_user <- user_starting_points[i]:user_endpoints[i]
    q_i <- q[movies_by_userId[current_user], , drop = FALSE]
    resids_i <- resids_by_userId[current_user]
    qi_by_qi <- crossprod(q_i) + lambda_pq_diag
    x <- crossprod(q_i, resids_i)
    p[u_rle$values[i], ] <- solve(qi_by_qi, x)
  }
  for (j in 1:n_movies) {
    current_movie <- movie_starting_points[j]:movie_endpoints[j]
    p_j <- p[users_by_movieId[current_movie], , drop = FALSE]
  }
}

```

```

    resid_j <- resid_by_movieId[current_movie]
    pj_by_pj <- crossprod(p_j) + lambda_pq_diag
    x <- crossprod(p_j, resid_j)
    q[m_rle$values[j], ] <- solve(pj_by_pj, x)
  }
  pq <- rowSums(p[user_int, , drop = FALSE] * q[movie_int, , drop = FALSE])

  loss <- rmse(resid - pq)
  delta <- abs(loss - prev_loss) / prev_loss

  if (delta < tol | delta > 1e07)
    break
  prev_loss <- loss
}

rownames(p) <- as.character(unique_users)
rownames(q) <- as.character(unique_movies)
list(
  mu = mu,
  b_i = as.data.frame(b_i_dt),
  b_u = as.data.frame(b_u_dt),
  b_g = as.data.frame(b_g_dt),
  p = p,
  q = q
)
}

fit <- fit_als_with_latent(train_set)
mu <- fit$mu
b_u <- setNames(fit$b_u$a, fit$b_u$userId)
b_i <- setNames(fit$b_i$b, fit$b_i$movieId)
b_g <- setNames(fit$b_g$c, fit$b_g$genres)
#
pq <- rowSums(fit$p[as.character(test_set$userId), ] *
              fit$q[as.character(test_set$movieId), ])
test_set$pq <- pq
resid <- with(test_set, rating - clamp(mu +
                                     b_i[as.character(movieId)] +
                                     b_u[as.character(userId)] +
                                     b_g[genres] +
                                     pq))
rmse(resid)

```

```
## [1] 0.8419696
```

One issue I encountered in an earlier version of the model was that around the 85th or so iteration, the RMSE would start growing exponentially until the `delta` and `loss` values became so large that R crashed entirely. I added a check so that if that occurred, the model would exit rather than crash. The cause was how I was calculating the latent factors. I was calculating `p` and `q` using the data table's `userId` and `movieId` columns instead of the indexing by user and movie, thus erroneously trying to calculate a latent effect per rating instead of per user and per movie. Creating vectors of unique user IDs and unique movie IDs, and then indexing movie ratings by user and user who gave ratings by movie, got me back on the right track.

The largest challenge was performance. Many earlier versions of the model used nested loops and fewer

indexes and took days—even at one point, just short of two weeks—to run. The key improvements to get it to run in a more reasonable amount of time were to calculate in two passes (first effects based on known factors, then latent effects) and to do as much pre-work as possible to index data in order to limit how many lookups needed to happen within the latent factor calculations. I also stopped using `rows_update()`, which I’d relied on previously to update the data table values in place.

Final Results

When tested against the 20% of data that had been held out throughout the model development, the RMSE was around 0.84, below the 0.86 target.

```
mu <- fit$mu
b_u <- setNames(fit$b_u$a, fit$b_u$userId)
b_i <- setNames(fit$b_i$b, fit$b_i$movieId)
b_g <- setNames(fit$b_g$c, fit$b_g$genres)

pq <- rowSums(fit$p[as.character(final_holdout_test$userId), ] *
              fit$q[as.character(final_holdout_test$movieId), ])
final_holdout_test$pq <- pq
resid <- with(final_holdout_test, rating - clamp(mu +
                                                  b_i[as.character(movieId)] +
                                                  b_u[as.character(userId)] +
                                                  b_g[genres] +
                                                  pq))

rmse(resid)

## [1] 0.8418162
```

While the model takes several minutes to run, it’s a vast improvement over the performance I was seeing earlier, and seems acceptable given the size of the dataset and that it’s being run on a fairly ordinary computer.

Conclusion

The combination of user, movie, genre, and latent effects improved the RMSE from a baseline of 1.059728 to 0.841691. That RMSE is validated by testing against a completely separate hold out test set that was not used for any part of the model development.

Limitations

While I am quite happy with my model, it does have limitations that could be improved upon in future iterations. Most notably, the model does not handle movies or users that were not in its original training set. I worked around this by assigning 20% of ratings per user to the test set and ensuring that if a user or movie had too few data points to be in both sets, it ended up in the training set. However, a more resilient model would account for unknown data.

Likewise, I made some simplifications to the model that make it less useful if given a differently sized training set. I used constant values for lambdas to make the values easier to comprehend and test, but that means a much smaller training set would be heavily overregularized. To make the model more flexible, it could be re-tuned using lambdas that are then multiplied by the total number of rows, now that the general values for a 7 million row set are known. Similarly, in earlier versions I attempted to set a minimum number of ratings to use for calculating latent effects, as the `fit_recommender_model` in the `dslabs` package does. I removed this as it added complexity to the calculations with little impact – since my dataset was large enough, the minimum ratings excluded very few, if any, rows. However, to support varying sizes of training data, this could be added.

Finally, given the length of time needed for the code to run, I did not try K values larger than 24. It is possible that including more latent effects might further improve the predictions.

Future Work

There are other ideas that I did not explore here that could be interesting to look it in a future model:

- I tried each unique combination of genres as a separate genre, since “Comedy|Horror,” for example, does seem to me to be a different experience from “Horror”. But that also means that it’s more difficult to tell whether “Horror” has any overall effect on ratings. Splitting the genres might yield different results
- After determining decade did not impact rating, I did not go back and test whether release year impacted rating. I did not expect that it would, but that could be validated
- While I tested known factors without latent effects, I did not do the reverse. My assumption was that the known factors should be included in the model, but a latent-effect-only model could also be worth trying

Acknowledgements

While I cannot cite every source I looked at online during the length of this project, there were a few resources that I consulted multiple times and to which I am particularly indebted.

- *Introduction to Data Science: Data Wrangling and Visualization with R* by Rafael A. Irizarry
- *Introduction to Data Science: Statistics and Prediction Algorithms Through Case Studies* by Rafael A. Irizarry
- “dslabs: Data Science Labs” R package, specifically the source code for `fit_recommender_model`
- “Claude” by Anthropic, Opus 4.6 and Sonnet 4.6 models